

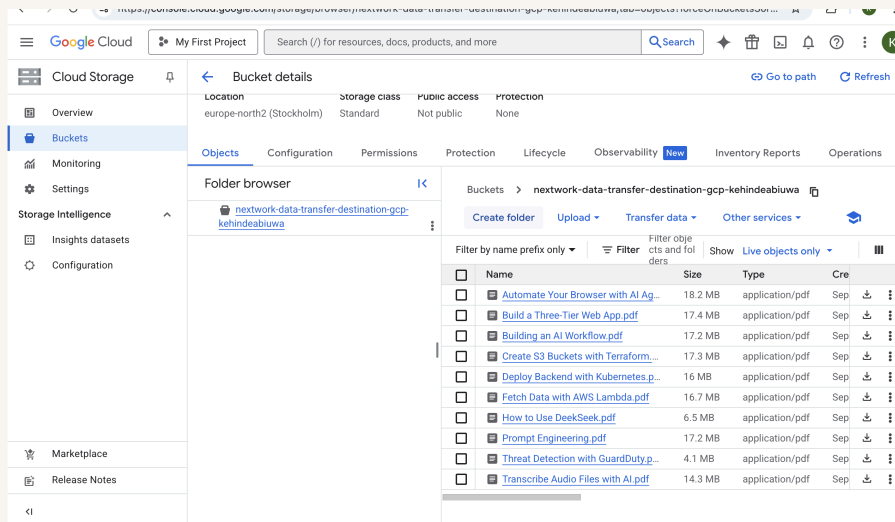


nextwork.org

Multi-Cloud Data Transfer with AWS and GCP



Kehinde Abiuwa





Introducing Today's Project!

In this project, I will demonstrate setting up S3 and GCS buckets, creating an AWS IAM role that GCP can assume (via external ID) to read S3, wiring S3 event notifications to Amazon SQS, and using those events to drive near-real-time transfers into GCS—plus troubleshooting the pipeline end to end, because I want hands-on practice with cross-cloud identity, secure data movement, and building low-latency, event-driven replication between AWS and GCP.

Tools and concepts

Services I used were Amazon S3, AWS IAM/STS, Google Cloud Storage (GCS), and Storage Transfer Service. Key concepts I learnt include cross-cloud identity federation (OIDC) with an IAM role and least-privilege access, choosing bucket region and storage class, batch vs event-driven transfers, using manifest files (TsvHttpData-1.0) to copy a subset, wiring S3 events → SQS → STS for near-real-time sync, and basic DR/backups, monitoring, and troubleshooting of transfer jobs.

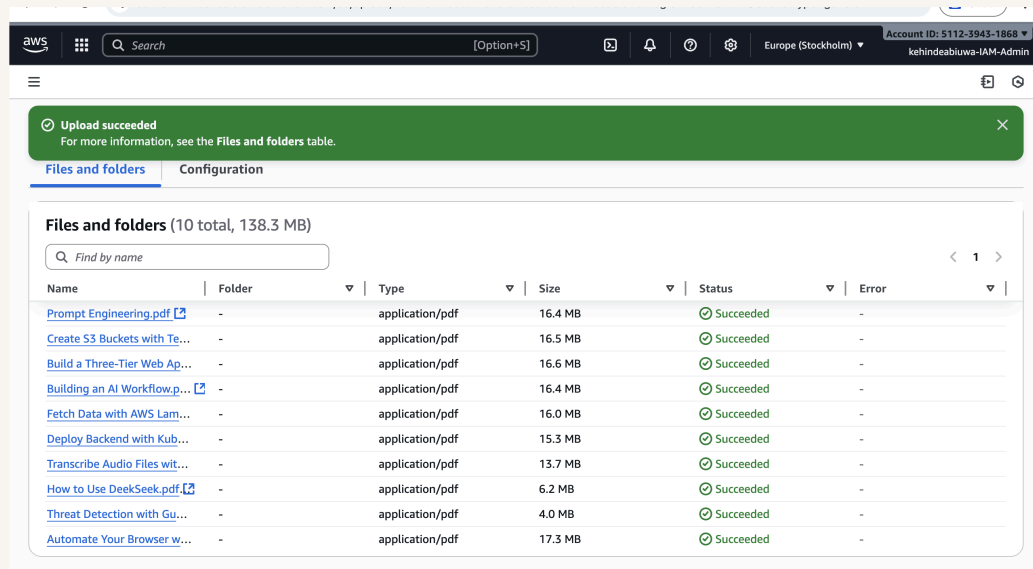
Project reflection

The most challenging part was wiring cross-cloud access—creating the AWS IAM role for identity federation, nailing the trust policy (subject ID), and scoping least-privilege S3 permissions—plus debugging a couple of AccessDenied/manifest header (TsvHttpData-1.0) hiccups. It was most rewarding to kick off the job and see my selected S3 objects land in GCS successfully, with the transfer marked Success and SQS events confirming the near-real-time flow.



Setting up Data in S3

I started this project by setting up a S3 bucket. I uploaded 10 nextwork project files that are in pdf format, You can literally upload almost any file format to a storage bucket.



Upload succeeded
For more information, see the Files and folders table.

Files and folders Configuration

Files and folders (10 total, 138.3 MB)

Find by name

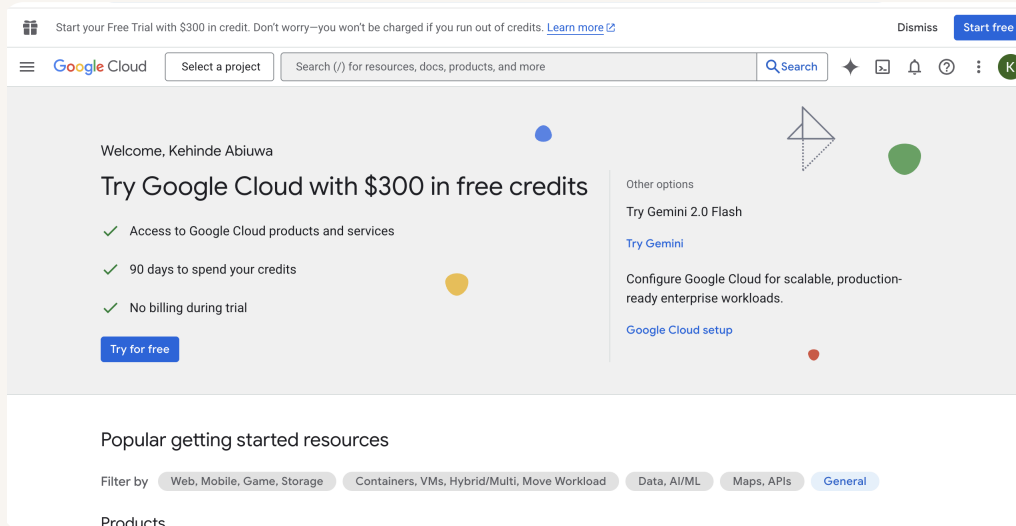
Name	Folder	Type	Size	Status	Error
Prompt Engineering.pdf	-	application/pdf	16.4 MB	✓ Succeeded	-
Create S3 Buckets with Te...	-	application/pdf	16.5 MB	✓ Succeeded	-
Build a Three-Tier Web Ap...	-	application/pdf	16.6 MB	✓ Succeeded	-
Building an AI Workflow.p...	-	application/pdf	16.4 MB	✓ Succeeded	-
Fetch Data with AWS Lam...	-	application/pdf	16.0 MB	✓ Succeeded	-
Deploy Backend with Kub...	-	application/pdf	15.3 MB	✓ Succeeded	-
Transcribe Audio Files wit...	-	application/pdf	13.7 MB	✓ Succeeded	-
How to Use DeekSeek.pdf	-	application/pdf	6.2 MB	✓ Succeeded	-
Threat Detection with Gu...	-	application/pdf	4.0 MB	✓ Succeeded	-
Automate Your Browser w...	-	application/pdf	17.3 MB	✓ Succeeded	-



Setting up GCP

Google Cloud Platform is Google's suite of cloud services (compute, storage, databases, networking, data/ML tools) you use on demand. I set up a GCP account with the purpose of receiving data from AWS S3 (via Storage Transfer Service or custom code) as part of my cross-cloud pipeline.

GCP's free tier includes Always Free usage limits on core services (like a small Compute Engine VM, Cloud Storage, Cloud Functions, Pub/Sub, and the BigQuery sandbox). I also get \$300 in credits to spend on any GCP products across the first 90 days of my account.





Storage Transfer

Data transfers are important for keeping data where apps and analytics need it, enabling backups/disaster recovery, and syncing sources (S3) with destinations (GCS) on a schedule or continuously. Using multiple cloud providers for the transfer has benefits such as vendor resilience, access to best-of-breed services on each cloud, broader regional coverage and reliability, cost optimization, and managed features like incremental syncing, retries, and monitoring.

The transfer is set up using Storage Transfer Service, which is a Google Cloud service designed for transferring data in and out of GCP Cloud Storage (which is like GCP's version of Amazon S3). We need this service because storage transfer service will handle authentication between the two platforms (AWS & Google Cloud), starts and stops the transfer, and makes sure data transfers correctly. Without it, I would need to build my own transfer system, which would mean downloading all the data to a computer first, then uploading it again to the other cloud - slow, expensive, and prone to errors!

There are two different types of transfers you could set up... The difference is Batch scheduling runs once or on a schedule (daily/weekly) to copy what's there—perfect for big migrations or periodic backups—while Event-driven fires automatically whenever a new or updated object appears at the source, giving near-real-time sync. Batch is simpler and great for catch-up jobs; Event-driven needs extra setup (notifications/IAM) but keeps destinations up to date almost instantly.





Granting GCP Access to AWS

Identity federation lets one identity system trust another. To connect AWS and GCP, I'm using identity federation, which works by creating an AWS IAM role that trusts a Google (OIDC) service account. When the Storage Transfer Service runs, GCP presents an OIDC token for that service account; AWS STS (AssumeRoleWithWebIdentity) exchanges it for short-lived credentials that can read my S3 bucket. This is more secure than other methods because there are no long-lived access keys to store, credentials are temporary and auto-rotated, permissions are least-privilege and easy to revoke, and access is fully auditable via IAM/STS.

I created a custom IAM role for GCP's Storage Transfer Service to assume in AWS via OIDC federation (`sts:AssumeRoleWithWebIdentity`), with a trust policy tied to my GCP service account's subject ID, because the transfer job needs temporary, scoped access to my S3 bucket without long-lived access keys and with clear, auditable security.

Within the IAM role, I needed to write a custom trust policy because I had to explicitly allow only my GCP Storage Transfer Service to assume the role via web identity and block everything else. The policy identifies GCS based on a subject ID, which is the immutable numeric unique ID of my project's Storage Transfer Service service account carried in the OIDC token's sub claim. Using the unique ID tightly scopes who can assume the role.



us-east-1.console.aws.amazon.com/iam/home?region=eu-north-1#/roles/create

Guest

Account ID: 5112-3943-1868
kehindeabiwa-IAM-Admin

Custom trust policy

Create a custom trust policy to enable others to perform actions in this account.

Custom trust policy

Create a custom trust policy to enable others to perform actions in this account.

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Principal": {  
7         "Federated": "accounts.google.com"  
8       },  
9       "Action": "sts:AssumeRoleWithWebIdentity",  
10      "Condition": {  
11        "StringEquals": {  
12          "accounts.google.com:sub": "116372634145241315189"  
13        }  
14      }  
15    }  
16  ]  
17 }
```

Edit statement Remove

Add actions for STS

Filter actions

All actions (sts:*)

Access level - read

☐ GetAccessKeyInfo info

☐ GetCallerIdentity info

☐ GetFederationToken info

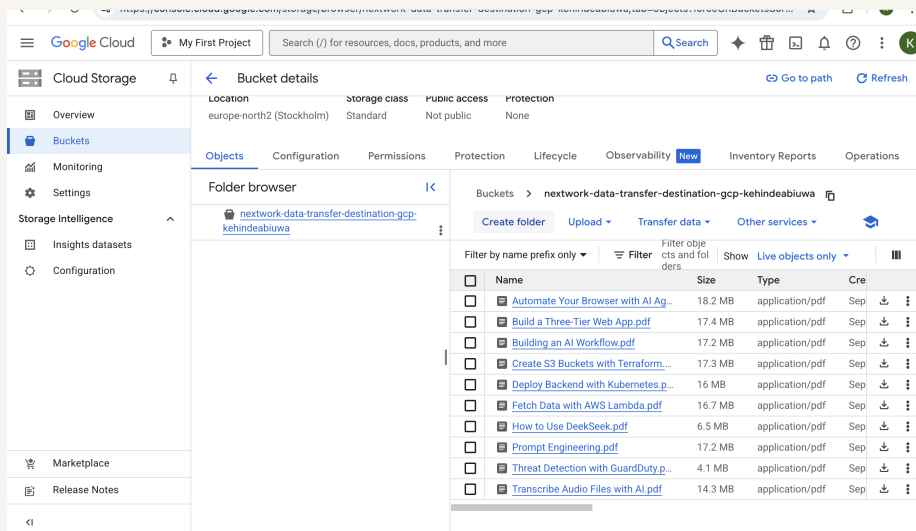
☐ GetFederationToken info



Transferring from S3 to GCS!

To set up my destination GCS bucket, I needed to set up its region, which means where the data physically lives and is served from (affects latency/residency/cost), and its storage class, which means the pricing tier based on access frequency (Standard for hot data; Nearline/Coldline/Archive for less frequent).

I verified my data transfer was successful by navigating into my google cloud storage bucket, and when I did that I saw the files transferred successfully from my S3 bucket into my google cloud bucket.

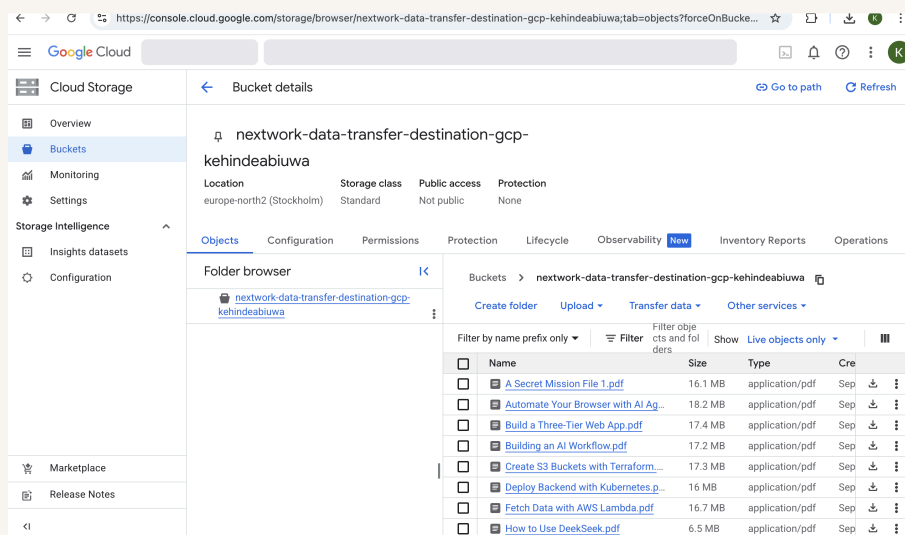




Transfer with a Manifest

In a project extension, I experimented with Storage Transfer Service manifests to copy only a subset of objects instead of an entire bucket. A manifest file works by acting like a shopping list: you add the required header TsvHttpData-1.0 at the top, then list the exact file paths/URLs you want transferred. STS checks the header to validate the format and then moves only the listed files—perfect for large buckets where you need a specific subset or repeatable, targeted runs.

I verified my data transfer was successful by navigating into my google cloud storage bucket, and when I did that I saw the files transferred successfully from my S3 bucket into my google cloud bucket.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

